

Monocular Depth using CARLA algorithm

Acey Dufresne

¹College of Software Engineering, Kennesaw State University

ABSTRACT

Depth estimation is transforming two dimensional input into three-dimensional output, monocular depth estimation is accomplishing this with one camera, eliminating the use of stereo vision, in which multiple cameras are used to mimic human eyes, and compare the inputs from the cameras to calculate the distance between the images. CARLA stands for 'Composite and Relighting algorithm.' The proposed algorithm will break down input into composite layers, organized into three sub-groups: "humans," "objects," and "environments." By using three groups the algorithm can generate relative depth, comparing sizes of different groups to one another. For example, in the dataset implementation of this project, the script already does this. Because the API currently being used doesn't use distinct specified measurements, the script currently assigns a 'head height,' which can then be given a random and arbitrary value in inches, depending on random variables such as gender, ethnicity, and age, these "head-units" can then be used to create ratios for the rest of the limbs and appendages. For example, the average person's arm, depending on age, is around three times the size of their head, and because the algorithm has applied a storage value to that head unit, we can assign a value for the arm, and continue in that manner. In the same way, we can use that process of relative depth between objects, to assign metric depth, or absolute depth to other objects. Once the algorithm has found the estimated mesh dimensions, the algorithm can UV project the initial frame texture back onto the model.

[Click here to view the in depth proposal:](#)

1. TOKENIZER AND DATASETS:

Currently the algorithm is using three datasets to learn from, as mentioned priorly, each of these datasets can be broken down into sub-sets as well. For example, the 'human' set, is compartmentalized into three sub-labels: base, clothed, and clothed+physics. Objects is split into several groupings, based on the corresponding API currently in use. The 'environmental' set is the set of HDRI's, which are environmental lighting, which are imperative to the models success, as the algorithm is rebuilding the input frame/image into a full three-dimensional environment. Therefore, the algorithm must rebuild everything from scratch, with the model textures being the only exception.

The tokenizer proposed is the BPE algorithm, which is most notably known for it's use in natural language processing. Byte-Pair Encoding works by finding the most frequent pair of input characters and merging those into one token. We can borrow this idea, and use the .fbx object mesh vector data as the input set. For example, if a mesh has this data: vertex 1: [0,1,0], vertex 2: [1,1,0], and vertex 3: [1,1,1], then the data for vertex one would: vector 1: [vector 1, vector 2, vector 3]. Combine a large quantity of these triangular meshes, and the model can begin to learn the patterns for arms, or legs, or any section of a body. We then use the same process for objects.

2. MULTI-CLASSIFIER AND DECODER:

A separate model is used to classify the input frames, the multi-classifier discovers elements within the input, and passes those labels to the decoder, which separates the input into chunks which it can then load into the corresponding GAN model.

3. GENERATIVE ADVERSARIAL NETWORKS:

Three models are trained, one for human meshes, one for object meshes. A third model is also used, an SNN, or Siamese Neural Network, which compares the input frame to three-dimensional view port, maneuvering the view port, and iterating through the environmental set until the two match.

3.1. Encoder and GAN:

The final step of the algorithm is 're-build' the input, the encoder receives the prediction meshes, and reconstructs the environment. The algorithm then uses a UV projection, to 'project' the initial frame input on the three-dimensional environment. However, this leaves an issue, while the visible portion of the input frame is now textured once more, the non-visible meshes are not. A final GAN model, trained on both 'human,' and 'objects,' datasets, goes through and populates the 'un-discovered' topology with texture.

Thus the entire model is modeled, and re-textured. Before the algorithm can pass along the output, the algorithm needs to re-mesh the topology. This consists of creating a new mesh, with far less vertices and topology, and then baking the original mesh textures into the new topology.

3.2. Long Term Goals:

The main goal is to reduce and eliminate the necessity for external API. If time permits, using Vulkan, or OpenGL, or C language buffer frames, create a custom three-dimensional renderer and view-port. This would also be beneficial with the SNN for classifying the HDRI, as in a custom view-port the algorithm could create it's own HDRI's. The benefit to using view-ports in general, is so that the model doesn't need to learn shading on base meshes, as traditional methods for scanning in environments rely on massive training sets which capture shading on the frame subjects. For example, Hydralite, which has been used on massive cinematic projects, but requires the "crazy-rig," which is a large Lidar setup, that surrounds the target, all of which is then scanned and used to train a deep learning model. Similiarly, PIFu is another method and model that has been used, but has the same issues. Neither of these datasets is accessible or open-sourced, and the hardware required to utilize them is expensive and un-wieldy.

Another goal is to introduce an understanding of physics to the model. For example, take 'Tesla Vision,' which is a stereo camera system used for their self-driving vehicles. These methods use cameras as a replacement for scanners, and an understanding of physical three-dimensional space to maneuver the vehicle. We can mimic this, while still only using one camera system. As mentioned above, the process of outputting the final prediction requires re-meshing the predicted object, in this process the algorithm has to use a method of shrink-wrapping a new mesh unto the old one. In doing this, we have a mesh with much less topology than the original prediction, and can use the normal maps to make inferences on surface detail. This lends itself well to creating a physical 'hitbox,' on elements in the input frame. The algorithm can generate the predicted mesh, and corresponding collision box, and use that to determine movement and object avoidance.